

NDNSF: A DATA-CENTRIC SERVICE FRAMEWORK FOR SECURE, COLLABORATIVE,
AND DYNAMIC NETWORKED APPLICATIONS

by

Tianxing Ma

A Dissertation Proposal

Submitted for the Committee's Review

Academic Advisor:

Dr. Lan Wang

Major: Computer Science

The University of Memphis

June 2026

Abstract

Modern networked applications increasingly run as collections of services across mobile devices, edge nodes, cloud resources, storage systems, and users. Existing host-centric frameworks provide useful tools, but their endpoint and channel model leaves service discovery, provider selection, channel security, caching, storage, and application state in separate systems. The problem is the mismatch between this model and applications that care about named services, named data objects, and named results rather than fixed endpoint connections.

Named Data Networking (NDN) offers named and secured data retrieval, in-network caching, multicast forwarding, and data-centric validation. These primitives match service-oriented applications in dynamic networks, but they still need to be organized into higher-level service functions: invocation, authorization, provider selection, provider collaboration, and large-data exchange.

This dissertation proposes the Named Data Networking Service Framework (NDNSF), a data-centric service framework for secure, collaborative, and dynamic networked applications. NDNSF v0.1 is the completed foundation: it provides a generic service API, NDN-based request/ACK/selection/response messages, selective provider choice, admission metadata, name-based access control, and MiniNDN evaluation against NSC and gRPC.

The remaining dissertation work extends this foundation toward collaboration among service providers such as drones, edge nodes, storage services, and inference accelerators. The planned framework mechanisms include targeted invocation for known providers, trusted local service composition inside one container, and large-data references for artifacts that should be retrieved by name instead of carried as service payloads. These mechanisms are intended to be reusable framework features rather than application-specific shortcuts.

The proposal organizes the remaining framework problem into four layers: invocation, authorization, coordination, and data handling. This structure separates facts about NDN primitives from the higher-level service mechanisms that NDNSF must provide.

Two validation workloads drive and evaluate the framework. The UAV workload tests NDNF under mobility, intermittent connectivity, command-sensitive services, video and telemetry data, and multi-UAV collaboration. The distributed AI inference workload tests NDNF under model retrieval, dependency execution, multi-provider cooperation, and large intermediate objects.

The central hypothesis is that combining NDN's data-centric communication model with explicit service-level abstractions can reduce the complexity of building secure and collaborative networked applications. The dissertation contribution is the design, implementation, and evaluation of NDNF; the UAV and distributed inference systems validate the framework and feed requirements back into it.

Keywords: Named Data Networking, service-oriented applications, data-centric security, UAV networks, distributed AI inference, provider collaboration

Table of Contents

Chapter	Page
1 Introduction	1
1.1 Motivation	1
1.2 NDN Building Blocks for NDNF	2
1.3 Research Gap	6
1.4 Thesis Statement	7
1.5 Research Questions	8
1.5.1 Expected Answer Shape	8
1.6 Expected Contributions	9
1.7 Proposal Scope Boundary	10
2 Frameworks for Service-Oriented Applications	12
2.1 Purpose of This Review	12
2.2 Host-Centric RPC and Microservice Frameworks	12
2.2.1 What These Systems Solve	13
2.2.2 Open Issues for NDNF	13
2.3 Robotics Middleware and Service Composition	14
2.3.1 Lessons for UAV Service Containers	14
2.4 Message Queues and Event Systems	15
2.5 NDN Computation and Service Frameworks	15
2.5.1 Function-Centric Computation	15
2.5.2 Serverless and Edge-Oriented NDN Frameworks	16
2.5.3 Remote Invocation Frameworks	16
2.5.4 Application-Specific NDN Frameworks	17
2.5.5 Security and Access-Control Mechanisms	18
2.6 NDN-Based Service Systems	18
2.7 Design Patterns and Trade-offs	19
2.8 Open Issues	19
3 NDNF Framework	21
3.1 Design Goals	21
3.2 NDNF v0.1 Baseline	21
3.3 Provider Collaboration as the Central Extension	24
3.4 Generic Dynamic API	24
3.5 Service Message Model	25
3.6 Service Invocation Workflow	26
3.7 Security Model	27
3.7.1 Rationale for ABE-Backed Service Authorization	29
3.7.2 Runtime Permission Withdrawal	31
3.8 Completed Foundation and Current NDNF Validation	31
3.9 Limitations of NDNF v0.1	34

3.10	Supporting Mechanisms for Provider Collaboration	35
3.10.1	Targeted Invocation	35
3.10.2	Trusted Local Invocation	36
3.10.3	Large-Data Reference Abstraction	36
4	UAV Network Application over NDN	38
4.1	Why UAV Networks Need a Service Framework	38
4.2	UAV Services	39
4.3	Architecture	40
4.4	Operator Workflow and State Discipline	41
4.5	Live Video and Recording	42
4.6	Flight Control and Safety	43
4.7	Multi-UAV Patrol	44
4.8	Evaluation Plan	44
5	Distributed AI Inference over NDN	46
5.1	Motivation	46
5.2	Approaches to Distributed AI Inference	46
5.3	NDN-Based Inference Architecture	47
5.4	Automatic Collaboration Plan Generation	49
5.5	Dependency-Driven Execution	49
5.6	Current Prototype Status and Evidence Boundary	50
5.7	Prototype and Evaluation Plan	51
6	Research Plan and Timeline	53
6.1	Completed Work	53
6.2	Planned Work	53
6.3	Evaluation Matrix	54
6.4	Timeline	54
6.5	Risks and Mitigation	54
7	Conclusion	57
	References	57

Chapter 1

Introduction

1.1 Motivation

Modern networked applications are increasingly organized as collections of services rather than as single monolithic programs. A mobile robot may expose telemetry, perception, and flight-control services. A ground station may provide object detection, mapping, mission planning, and storage services. Edge devices may request AI inference services from nearby accelerators or cloud nodes. In each case, the application developer wants to invoke named services and retrieve named results without manually binding every operation to a fixed host address, a long-lived channel, and a single provider.

Today's service frameworks provide valuable engineering tools, but TCP/IP communication is still centered on reaching endpoints and maintaining channels. Remote procedure call (RPC) systems, REST services, and microservice platforms typically identify service endpoints by host addresses, ports, or channel endpoints. Robotics middleware provides publish-subscribe or request-response communication, but often assumes an operational domain where identity, trust, network reachability, and service discovery are configured outside the service invocation itself. The problem is not that these tools are unusable; the problem is that mobile and data-heavy applications want to name the service and the data objects involved in the workflow, while the networking substrate mainly exposes where an endpoint is and how to keep a path to it. When applications become mobile, intermittent, multi-provider, and security-sensitive, developers must rebuild mechanisms for discovery, retry, authorization, integrity verification, data retrieval, and provider selection.

Named Data Networking (NDN) offers a different foundation. NDN names data directly, secures Data packets directly, and decouples retrieval from fixed endpoints. These properties naturally match service-oriented applications in which users care about a named service, a named model artifact, a named telemetry object, or a named video segment rather than a particular

IP connection. However, NDN by itself is a networking architecture, not a complete service framework. Application developers still need a runtime that defines how to publish service requests, authorize users and providers, select one or more providers, protect service data, and support higher-level collaboration.

This gap also emerged from the application-driven development in this dissertation. Building the UAV application exposed the need for stable service selection, command authorization, telemetry freshness, local service composition, and large video/recording object handling. Building the distributed inference workload exposed the need to coordinate artifact provisioning, tensor dependencies, provider roles, and large intermediate data. These experiences motivate treating the gap as a framework problem rather than as isolated application glue code.

1.2 NDN Building Blocks for NDNSF

This proposal assumes several NDN concepts, but they are used here for a specific purpose: to explain why NDNSF is a data-centric service framework. NDN is receiver-driven: a consumer sends an Interest packet for a named object, and the network returns a matching Data packet from a producer, cache, or other available source [1]. This basic exchange is not yet a complete service invocation, but it supplies the communication primitive that NDNSF builds on: the application names what it wants, not the host that should send it; the returned Data packet is independently named and signed; and retrieval can succeed even when the original producer is not on the same path as the consumer.

NDN names have three roles that are important for this dissertation. First, they identify application data, such as telemetry, video segments, model artifacts, service requests, ACKs, selections, and responses. Second, they are part of the security model: trust schemas and access-control attributes can be defined over name prefixes. Third, they guide routing and forwarding because Interests are forwarded by name prefix rather than by destination address. NDNSF uses the same naming foundation for service-level objects, so a service workflow can refer to both actions and large data by name.

NDN forwarders maintain three data structures that are important for NDN's design. The Forwarding Information Base (FIB) forwards Interests by name prefix. The Pending Interest Table (PIT) records unsatisfied Interests and aggregates multiple Interests for the same name, allowing one returned Data packet to satisfy multiple consumers. The Content Store (CS) caches Data packets for future retrieval. Together, these structures support stateful forwarding, in-network caching, multicast-style delivery, and adaptive retrieval under mobility or failures [2]. For service applications, this means that service-related data can be named and retrieved independently of a fixed address or connection. Service invocation semantics, however, still have to be built above these packet and forwarding primitives.

NDN is data-centric in its security model as well as in its naming and forwarding model. Each Data packet carries a producer signature, allowing consumers to verify integrity and producer authenticity regardless of whether the Data came from the original producer, an in-network cache, or a repository [1]. This property is central to NDN: service requests, ACKs, selections, responses, permissions, and large artifacts can all be treated as named and verifiable objects. However, signatures alone do not decide who is allowed to call a service, which provider is authorized to offer it, or which attributes should protect each service message. Those higher-level decisions are framework responsibilities.

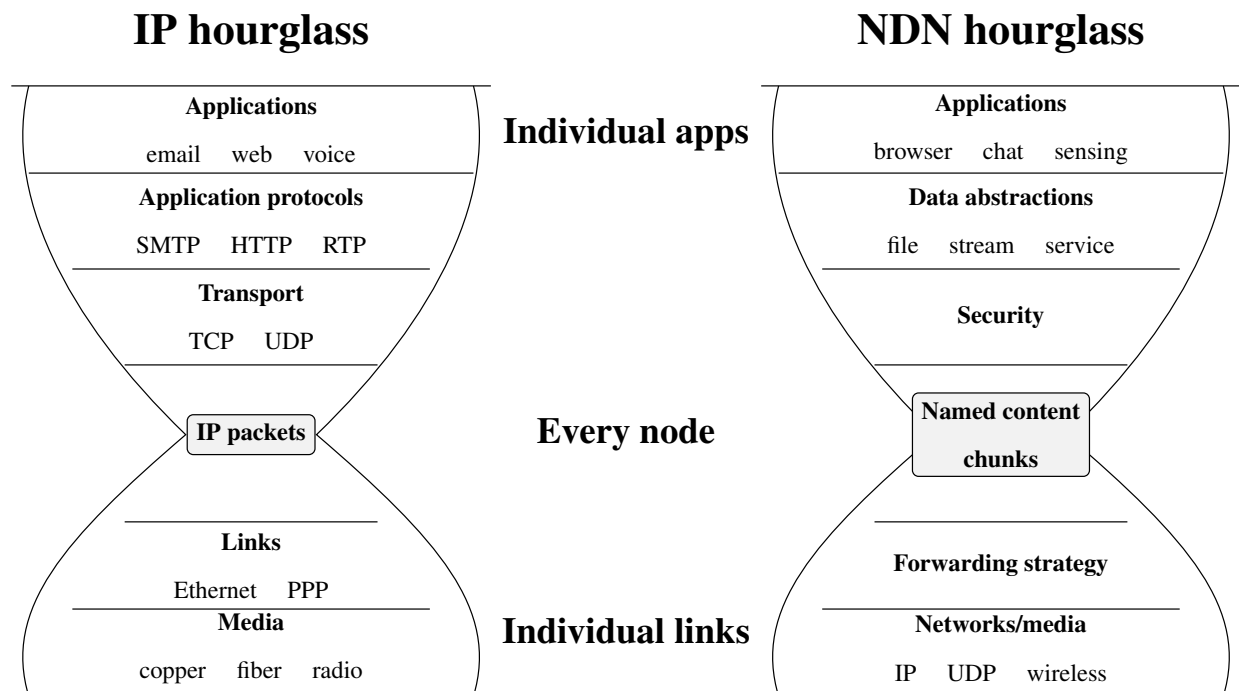


Figure 1: Conceptual comparison of IP and NDN hourglass models, redrawn from the NDN architecture discussion [1].

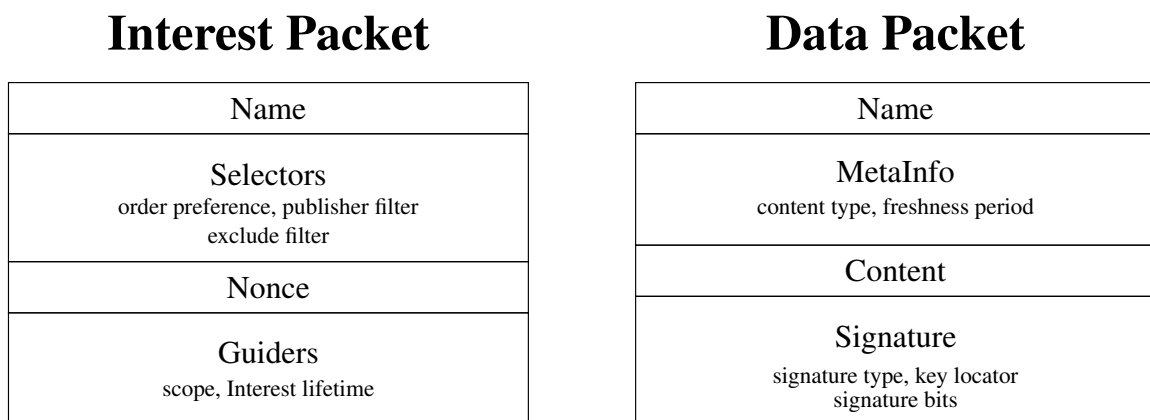


Figure 2: Simplified NDN Interest and Data packet structure, redrawn from the NDN architecture paper [1].

Receiver-driven retrieval and in-network caching are useful, but service invocation needs more than fetching a single object. Multiple providers may need to learn that a request has been published, users may need to collect ACK metadata before selecting providers, and long-running work may need status diagnosis. NDNSF therefore uses NDN Sync, especially State Vector Sync as implemented by NDN-SVS, to let users and providers discover newly published service

messages without binding the service to one fixed endpoint [3–5]. In NDN, Sync is not the service semantics itself; it is the dissemination substrate over which the framework publishes Request, ACK, Selection, and Response messages as named Data.

The implementation is built on the standard NDN software stack. NFD provides the local forwarder and face abstraction, while `ndn-cxx` provides the C++ application library for names, packets, faces, signing, validation, and segmented object retrieval [6]. These libraries allow NDN to use ordinary NDN names and packets rather than inventing a separate transport format. They also make it possible for applications to pass large artifacts by reference: models, video recordings, images, logs, and intermediate inference data can be segmented into named Data packets, signed, optionally encrypted, retrieved by name, and verified outside the small service-request payload.

For access control, NDN builds on NAC-ABE, which combines NDN naming with attribute-based encryption so that service-level authorization can be expressed over names rather than transport connections [7]. NDN uses this capability together with controller-issued permissions, service-level attributes, and per-invocation tokens. The result is that NDN supplies the data-centric communication substrate, while NDN supplies the service-layer runtime: who may call a service, who may provide it, how providers are selected, how messages are protected, and how large named objects are attached to service workflows.

Table 1: How NDN building blocks motivate NDN-SF mechanisms.

NDN Building Block	NDN-SF Mechanism
Data packets	Request, ACK, Selection, and Response are service message objects carried as named Data.
Interests	Interests fetch named Data; NDN-SVS uses Sync Interests to announce dataset updates.
Data signatures and name-based trust	Service messages, permissions, and large objects can be verified independently of the path or cache that returned them.
FIB/PIT/CS forwarding and caching	Service-related data can be retrieved by name and remain meaningful across mobility, caching, and intermittent connectivity.
NDN Sync / SVS	Users and providers discover newly published service events without binding a service to a fixed endpoint.
Segmented Data retrieval	Large models, videos, logs, and intermediate tensors are carried by reference rather than embedded in small service payloads.
NAC-ABE and named attributes	Service-level permissions and confidentiality policies are expressed over names and attributes rather than transport channels.

1.3 Research Gap

The research gap is therefore between NDN communication primitives and service-oriented application semantics. NDN provides the right lower-level building blocks: named Data, signatures, caching, forwarding state, segmented retrieval, and Sync-based discovery. A service framework must organize these building blocks into a consistent higher-level runtime.

This proposal structures the gap into four layers. At the *invocation layer*, a user needs

to publish a service request, collect ACK metadata, and select one or more providers. At the *authorization layer*, users and providers need service-level permissions, name-based attributes, replay protection, and verifiable responses. At the *coordination layer*, multiple providers may need to advertise readiness, cooperate on one task, or compose local and remote services. At the *data layer*, services need to attach large objects such as video, model files, logs, and intermediate tensors by name rather than carrying them inside small request payloads.

NDNSF addresses this layered gap by treating service invocation, authorization, provider selection, collaboration, and large-data reference handling as reusable framework mechanisms rather than application-specific glue code.

1.4 Thesis Statement

This dissertation argues that dynamic service-oriented applications expose a recurring mismatch between TCP/IP’s connection-oriented model and service-oriented applications’ data-centric needs: applications want to operate on named services, named data, and verifiable results, while the network stack usually exposes addresses, channels, and endpoints. A data-centric service framework can close this gap by making service invocation, authorization, provider selection, collaboration, and large-object retrieval explicit framework-level abstractions over NDN names and verifiable Data packets.

NDNSF v0.1 is a completed core contribution of this dissertation and demonstrates the feasibility of this direction. The remaining dissertation work extends this foundation into a framework for real dynamic service applications and evaluates it through two high-pressure domains with different service requirements:

- **UAV network applications** represent mobile, command-sensitive, real-time service-container workloads, where intermittent connectivity, command authorization, telemetry, live video, recording, object detection, and multi-UAV missions must be coordinated across a ground station and multiple drones.

- **Distributed AI inference** represents artifact-heavy, multi-stage compute/data collaboration workloads, where model shards, runtime artifacts, intermediate tensors, large-data objects, and heterogeneous inference providers must be selected, verified, and coordinated.

1.5 Research Questions

The dissertation is organized around the following research questions.

- RQ1. What problems in host-centric service frameworks can be addressed by data-centric service frameworks?
- RQ2. How should NDN SF expose service invocation, authorization, provider selection, collaboration, and large-object retrieval as generic framework abstractions without forcing each application to design a custom NDN protocol?
- RQ3. What reliability, security, and latency tradeoffs arise when NDN service requests are published to multiple providers and resolved through ACK metadata, provider selection, and collaboration, and how should supporting mechanisms such as Targeted invocation and compensation be used?
- RQ4. How can UAV network applications evaluate NDN SF under mobile, command-sensitive, real-time service-container workloads?
- RQ5. How can distributed AI inference evaluate NDN SF under artifact-heavy, multi-stage compute/data collaboration workloads?

1.5.1 Expected Answer Shape

The dissertation is expected to answer these questions in the following form. RQ1 should identify a set of recurring service-application problems that are not merely implementation gaps in existing systems, but consequences of binding services to hosts, sessions, or channels. RQ2 should show that NDN SF can expose service invocation, authorization, provider selection, and

provider collaboration as reusable framework mechanisms, with local composition and large-data references serving as supporting abstractions rather than application-specific NDN protocols. RQ3 should characterize when one-to-many request publication, ACK metadata, provider selection, and provider collaboration improve reliability, and when supporting mechanisms such as Targeted invocation or compensation are needed to control latency and recovery costs without weakening security checks. RQ4 should show that UAV workloads exercise the same framework mechanisms under mobility, command sensitivity, real-time video, and multi-drone collaboration. RQ5 should show that distributed inference exercises the same mechanisms under artifact provisioning, tensor dependency exchange, provider role assignment, and long-running distributed execution.

1.6 Expected Contributions

The expected dissertation contributions are:

1. A review and gap analysis of service-oriented application frameworks with a focus on why dynamic, secure, collaborative applications still require a new data-centric framework.
2. The design and implementation of NDN-SF as a generic dynamic C++ service framework over NDN.
3. Extensions to NDN-SF centered on provider collaboration, with supporting mechanisms for targeted invocation, trusted local service composition, and large-data references.
4. A UAV service-container application over NDN-SF, including flight control, telemetry, video, encrypted recording, object detection, and collaborative patrol services.
5. A distributed AI inference application over NDN-SF, including model splitting, service plans, reference-based artifact exchange, and automatic collaboration-plan generation.
6. An evaluation of NDN-SF and the application prototypes using MiniNDN, SITL-based UAV experiments, and selected real-machine validation.

1.7 Proposal Scope Boundary

For clarity, this proposal treats NDNFSF v0.1 as a completed core contribution of the dissertation, not as the final complete system. NDNFSF v0.1 establishes the basic architecture and feasibility of a data-centric service framework. The remaining dissertation work extends this foundation into an application-validated framework. The main framework extension is provider collaboration: how multiple authorized providers discover requests, advertise capabilities, are selected, exchange state or artifacts, and jointly complete a service task. Targeted invocation, trusted local invocation, and large-data references are treated as supporting mechanisms that make this collaborative service model practical under low-latency, same-process composition, and large-object workloads. These mechanisms are not UAV-only or AI-only features; they are proposed NDNFSF framework mechanisms whose usefulness will be evaluated through two demanding application classes.

The core dissertation question is therefore not whether two applications can be built, but whether NDNFSF can provide reusable data-centric service mechanisms that remain valid across two very different workloads.

The UAV application is used to validate NDNFSF as a service-container framework for mobile, command-sensitive, real-time systems. Its design exercises typed telemetry, mission, video, and safety information; Targeted flight-control commands; adaptive live video; encrypted local recording; object-detection service composition; and multi-drone patrol workflows. The DistributedInference application is used to validate NDNFSF as a framework for artifact-heavy, multi-stage compute/data collaboration. Its design exercises automatic model plan generation, reference-based model and activation artifacts, predictable prefetchable object names, and dependency-driven execution over tensor DAGs. The dissertation therefore does not present UAV and DistributedInference as isolated demos; it uses them as two complementary stress tests for the same data-centric service framework thesis.

These applications also contribute requirements back to NDNFSF. The UAV workload stresses low-latency command invocation, structured command status, service-container

composition, and large-data video references, while the DistributedInference workload stresses dependency-driven collaboration, artifact references, provider role assignment, and long-running selection diagnosis. Thus the application work has two roles in the dissertation: it is valuable in its own domains, and it is the primary method for validating and refining provider collaboration as the central NDNFS extension.

Chapter 2

Frameworks for Service-Oriented Applications

2.1 Purpose of This Review

This chapter is not intended to be a broad survey of every service framework. Instead, it reviews what existing systems have already achieved and identifies the open issues that motivate NDN. The key question is not whether systems such as gRPC, ROS, microservices, or NDN Service Call are useful; they clearly are. The key question is why dynamic, secure, collaborative applications still require another framework despite the existence of those systems.

2.2 Host-Centric RPC and Microservice Frameworks

RPC systems such as gRPC [8], HTTP/REST systems [9], and microservice platforms have become the dominant way to build distributed software. They provide typed APIs, transport bindings, load balancing, service registration, and production tooling. These systems work very well when services can be identified by stable endpoints, when channels can be established and maintained, and when security can be expressed as channel security plus service-side authorization.

However, the abstraction becomes less natural for applications with mobility and intermittent connectivity. A user of a UAV video service may care about the latest frames from drone A, not whether drone A is currently reachable through a particular IP address. A distributed inference client may care about the service name for a model stage, not a fixed GPU server. A mission-planning application may need any available drone that satisfies a role, not exactly one preconfigured endpoint. Host-centric frameworks can support these scenarios, but the burden shifts to external discovery, retry, state tracking, and application-specific coordination.

2.2.1 *What These Systems Solve*

RPC and microservice systems solve several problems that NDNFSF should not ignore. First, they give developers a familiar programming model: a service has an interface, a request type, a response type, error handling, and often a deadline. Second, production systems around gRPC and REST provide mature tooling for load balancing, logging, monitoring, deployment, and versioning. Third, channel security such as TLS gives a practical and widely deployed way to protect client-server communication. For stable cloud services, these features are often sufficient.

NDNFSF does not attempt to replace this engineering knowledge. Instead, it asks which assumptions break when service providers are mobile, intermittently connected, replicated, or embedded in resource-constrained edge environments. In these settings, a service call is not only a channel operation. It is also a decision about which provider is currently authorized, reachable, capable, and worth selecting. Existing host-centric frameworks can implement this behavior, but the logic is typically distributed across service discovery systems, load-balancers, application retries, middleware plugins, and service-specific policy code.

2.2.2 *Open Issues for NDNFSF*

The first issue is endpoint dependence. Even if modern service meshes hide IP addresses behind logical service names, the underlying communication still depends on reaching an endpoint through host-centric routing. The second issue is data lifetime. Once a response, video chunk, model shard, or telemetry log is stored outside the original channel, applications must add their own verification and encryption model. The third issue is provider multiplicity. RPC normally selects a single server before the request is sent; NDNFSF instead allows multiple providers to observe a request, decide whether to acknowledge it, and expose metadata that helps the user choose. These differences motivate a data-centric service framework rather than another IP endpoint wrapper.

2.3 Robotics Middleware and Service Composition

Robotics middleware, including ROS and ROS2 [10, 11], provides a more application-aware model for robot components. Nodes can publish topics, call services, and interact through standardized message types. ROS2 improves quality-of-service control and deployment flexibility compared with earlier ROS systems. These frameworks are important precedents for UAV applications because they treat sensors, control, telemetry, and perception as composable modules.

The remaining gap is that robotics middleware is not a general data-centric network security framework. Trust anchors, certificate deployment, data-at-rest protection, multi-provider service selection, and cross-domain service authorization are not the main abstractions. In addition, a UAV application deployed across a ground station, multiple drones, and intermittent wireless links still needs naming, verification, and recovery mechanisms that are robust outside a single local robotics domain.

2.3.1 *Lessons for UAV Service Containers*

ROS is particularly important for this dissertation because it shows that robotics applications are naturally service-oriented. A UAV system is not a single communication flow. It contains flight-control commands, telemetry streams, camera frames, mapping, mission planning, object detection, logging, and health monitoring. ROS-style composition validates the idea that UAV software should be decomposed into services.

However, NDN-SF targets a different layer. ROS and ROS2 are strong middleware systems for local and distributed robot components, while NDN-SF focuses on named, secure, network-visible service invocation over NDN. In the proposed UAV application, the drone app can still be viewed as a service container, but external interactions such as ground-station commands, telemetry discovery, video retrieval, and cross-drone collaboration use NDN-SF service names and data-centric verification. This distinction is important: NDN-SF is not a replacement for flight controllers or robotics middleware, but a network service framework that

can connect such components across dynamic NDN environments.

2.4 Message Queues and Event Systems

Message queues and publish-subscribe systems decouple producers and consumers in time [12]. They are useful for telemetry, logs, asynchronous workflows, and elastic back-end processing. This is relevant to NDN because many service applications include both request-response actions and event streams. For example, a drone may receive a command as a service request while publishing telemetry or video metadata asynchronously. A distributed inference pipeline may publish intermediate results and execution events while serving user requests.

However, many message queue deployments rely on brokers or logically centralized infrastructure. Security is often enforced by broker access control and channel encryption rather than by data objects that remain verifiable wherever they are stored or forwarded. For mobile or disconnected edge environments, broker availability and placement can become a limiting assumption. Moreover, queues do not by themselves define how a user should choose among multiple providers, verify that a provider is authorized to serve a specific service, or retrieve large service artifacts as independently verifiable data.

2.5 NDN Computation and Service Frameworks

NDN already provides useful primitives for service-oriented systems: hierarchical names, receiver-driven retrieval, in-network caching, adaptive forwarding, and data signatures [1]. A service framework over NDN must bridge the gap between these primitives and application-level service semantics. Prior NDN work explores several regions of this design space.

2.5.1 Function-Centric Computation

Named Function Networking (NFN) treats computation as a first-class named object by embedding function expressions in names [13]. This model is powerful because it allows

computation and data retrieval to be composed within the naming system itself. It also suggests a deep integration between content names and computation names, which is attractive for in-network processing.

The trade-off is complexity. Function-centric naming can make names highly expressive, but it also makes execution semantics, trust, caching, and deployment more difficult to reason about. UAV control, live video, and distributed AI inference require explicit service authorization, provider selection, long-running state, and application-defined execution policies. NDN SF therefore chooses a service-invocation abstraction rather than a fully general function-expression abstraction.

2.5.2 Serverless and Edge-Oriented NDN Frameworks

NFaaS extends NDN toward serverless computing by treating functions as deployable units that may be executed at edge locations [14]. Other edge-oriented frameworks, such as CFEC for information-centric Internet of Vehicles, explore low-latency microservice execution and dynamic placement in edge environments [15]. These systems highlight an important direction: service execution should move closer to users, data sources, or resource-rich edge nodes.

The open issue is that dynamic placement is only one part of service-oriented application support. Applications also need a stable API, authorization for both users and providers, multi-provider selection, large-object handling, and clear deployment procedures. NDN SF focuses on these runtime service semantics. It can coexist with future placement mechanisms; for example, a placement algorithm could decide which drones, storage components, or inference providers should advertise a given NDN SF service.

2.5.3 Remote Invocation Frameworks

RICE provides remote method invocation in ICN using mechanisms such as handshakes and thanks to represent computation parameters and results [16]. NSC proposes Named Service

Calls for NDN and offers an RPC-like abstraction for remote execution [17]. These systems are close to NDN SF's goals because they directly address service invocation rather than only content retrieval.

The key limitation is that they do not provide the complete set of mechanisms needed by the application classes targeted in this dissertation. NSC is primarily one-to-one: a caller contacts an executor, and the protocol is not designed around simultaneous provider discovery, provider ACK metadata, or multi-provider selection. RICE provides stronger invocation semantics but requires changes to the forwarding plane. NDN SF keeps the framework at the application/runtime layer and uses NDN Data publication and Sync to support one-to-many service request dissemination without modifying NFD.

2.5.4 Application-Specific NDN Frameworks

Other NDN systems focus on particular domains. DNMP builds a secure network measurement framework using NDN synchronization and hierarchical topics [18]. MIA-NDN explores microservice-centric Interest aggregation for NDN [19]. SECaaS studies security-as-a-service in multi-access edge environments [20]. These projects demonstrate that NDN can be adapted to practical service-like workloads, especially when names and application semantics are designed together.

The limitation is generality. Application-specific frameworks often make assumptions about a particular data type, service domain, aggregation pattern, or deployment topology. NDN SF instead aims to provide a reusable framework layer that can support different application services, including UAV control, video, object detection, distributed inference, storage, and collaboration. The proposed UAV and DistributedInference applications are intended to test whether the framework is general enough for multiple domains rather than only one carefully tuned application.

2.5.5 *Security and Access-Control Mechanisms*

NDN signs Data packets by design, but signatures alone do not answer all service questions. A service framework must decide who may request a service, who may provide a service, what data should be encrypted, how keys should be distributed, and how replayed messages should be rejected. Prior work on attribute-based encryption and NAC-ABE shows how fine-grained access control can be integrated with NDN data names [7, 21]. These ideas are central to NDNSF.

The open issue is integration with the service workflow. If access control is bolted onto individual applications, every application must separately handle permission retrieval, encryption attributes, message verification, and replay protection. NDNSF makes these part of the runtime: ServiceController distributes permissions, service messages are protected according to service-level attributes, and one-time user/provider tokens bind ACK, selection, targeted fast-path, and response messages to the current invocation.

2.6 **NDN-Based Service Systems**

NDN Service Call (NSC) [17] and other NDN application frameworks show that NDN can support service invocation without IP endpoint binding. NDN's Interest/Data exchange, name-based routing, in-network caching, and data signatures address several issues in IP-based systems. These systems are therefore important evidence that NDN is a promising foundation for services.

The remaining gap is that existing NDN service systems have not provided a general-purpose runtime that combines:

- generic dynamic service APIs,
- controller-authorized user and provider permissions,
- one-to-many request publication,

- provider ACK metadata,
- application-defined selection strategies,
- adaptive admission control,
- large-data references,
- provider collaboration, and
- service containers that can be evaluated in realistic settings.

2.7 Design Patterns and Trade-offs

The related work suggests several recurring design patterns. First, service frameworks must decouple service names from provider locations. Second, large inputs and outputs should be represented as named data objects rather than embedded directly in small request payloads. Third, service state usually cannot rely only on NDN’s Pending Interest Table because service execution may be long-running, multi-step, or collaborative. Fourth, security must be part of the invocation workflow rather than an afterthought.

These patterns also expose trade-offs. Function-centric names are expressive but can be difficult to deploy. Serverless placement is flexible but requires orchestration. RPC-style invocation is easy to understand but can favor one-to-one execution. Broker-based systems decouple time but may introduce infrastructure dependencies. NDN’s design point is to combine an RPC-like developer API with NDN’s data-centric names, signed Data, Sync-based publication, and application-visible provider selection.

2.8 Open Issues

Table 2 summarizes the open issues motivating NDN.

Table 2: Open issues in existing service-oriented frameworks.

Issue	Why It Matters
Endpoint binding	Mobile services and intermittently connected providers should be invoked by name and capability rather than by stable host address.
One-to-many invocation	Reliability and collaboration often require multiple providers to hear a request and allow the user to select one or more of them.
Data-centric security	Data should remain signed, verifiable, and optionally encrypted at rest, not only while moving through a secure channel.
Provider authorization	A user must verify not only that it is allowed to call a service, but also that the selected provider is authorized to offer it.
Provider selection	Provider metadata such as load, queue size, location, and availability should be part of the service workflow.
Large data	Models, video, images, logs, and intermediate tensors should not be forced into small request payloads.
Collaboration	Tasks may require multiple providers, compensation requests, or aggregation of partial results.
Operational validation	Real systems need repeatable certificate, routing, configuration, and experiment setup to validate the framework outside toy examples.

NDNSF is designed to address these issues using NDN names, verifiable Data, sync-based publication, and service-level authorization.

This chapter supports RQ1 by turning the related-work review into a set of framework requirements rather than a survey checklist. It also prepares RQ2 and RQ3 by identifying which properties must be provided by the NDNSF runtime instead of being reimplemented separately by each UAV or inference application.

Chapter 3

NDNSF Framework

3.1 Design Goals

NDNSF is designed around five goals.

1. **Application-facing simplicity.** Developers should register and invoke typed services without writing service-specific NDN protocols.
2. **Data-centric security.** Service messages and permissions should be signed, verifiable, and encrypted where appropriate.
3. **Dynamic provider selection.** A request should be able to reach multiple providers and select the best one according to strategy.
4. **Collaboration and resilience.** The framework should support multiple providers, compensation after partial failure, and cross-validation.
5. **Operational validation.** The framework should support MiniNDN evaluation, selected real-machine experiments, certificate management, and reproducible configuration.

3.2 NDNSF v0.1 Baseline

NDNSF v0.1 includes:

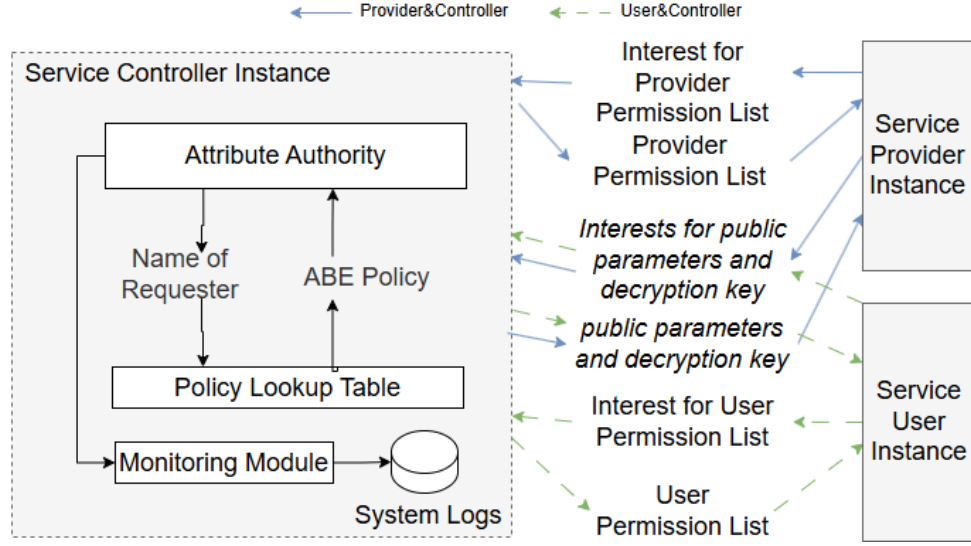
- a generic dynamic C++ service API;
- ServiceController, ServiceUser, and ServiceProvider runtimes;
- service request publication over NDN synchronization;
- Request, ACK, Selection, and Response message flow;
- user and provider permission distribution;

- NAC-ABE-based service-level access control;
- selective ACK and provider metadata support;
- built-in provider-selection strategies;
- adaptive admission control;
- an NDN-SVS patch used by the evaluation path;
- Python bindings for experiment and workload integration;
- MiniNDN evaluation against NSC and gRPC.

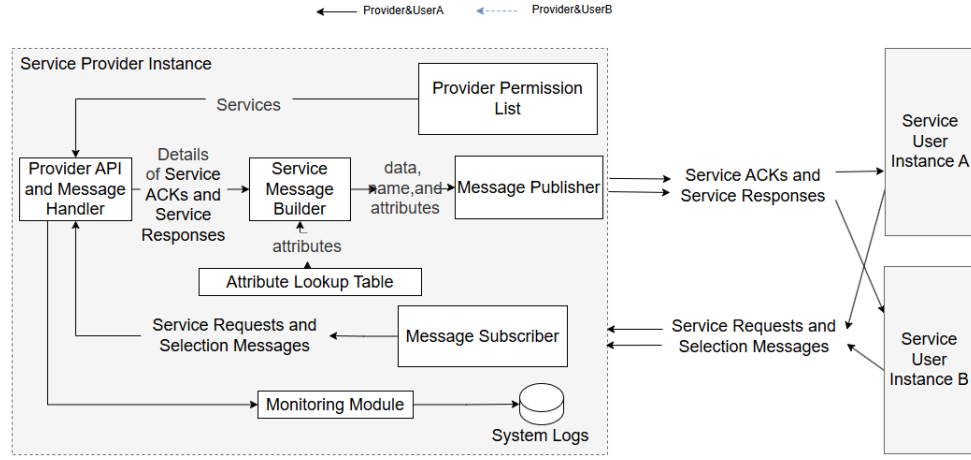
This baseline is substantial because it already connects the main pieces that a data-centric service framework must coordinate: service naming, service-message publication, user and provider authorization, encrypted service payloads, provider metadata, provider selection, and response verification. In other words, NDN v0.1 is not only an API sketch; it is an executable runtime that shows how NDN names, Sync, signed Data, NAC-ABE attributes, and service-level policy can be combined into one service-invocation workflow. This is why the proposal can focus on extending and validating the framework rather than starting from a blank design.

A paper describing this completed foundation has been submitted to ICNP. The proposal therefore treats NDN v0.1 as prior completed dissertation work and uses the remaining chapters to define how that foundation will be extended and validated through application-driven workloads.

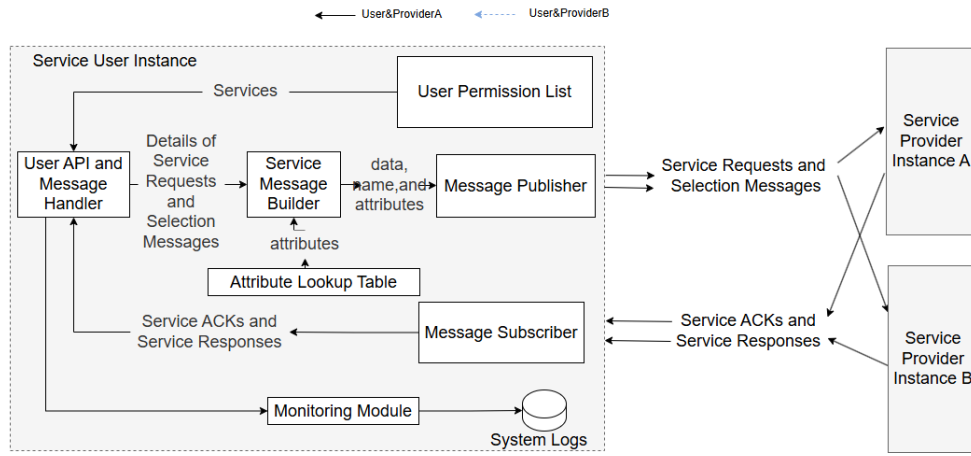
Figure 3 summarizes the three runtime roles implemented in the baseline. The controller manages policies and distributes permissions, providers subscribe for authorized requests and publish service results, and users publish requests, collect ACK metadata, select providers, and verify responses. This role separation is the foundation for the later UAV and distributed-inference service-container work.



(a) Controller



(b) Provider



(c) User

Figure 3: NDNSF v0.1 runtime roles used as the proposal baseline.

3.3 Provider Collaboration as the Central Extension

The central extension beyond NDN v0.1 is provider collaboration. In a collaborative service, one logical request may involve multiple authorized providers rather than a single endpoint. Providers may advertise ACK metadata such as availability, queue state, location, or role capability; the user may select one or more providers after an ACK collection interval; and the application may issue compensation requests when only part of a logical task is completed. Long-running selected work should also be diagnosable: after a timeout, the user should be able to learn whether a selected provider has not seen the selection, is queued, is executing, has failed, or has already published a response.

This collaboration model is intentionally application-independent. Multi-UAV patrol can use it to assign mission segments and recover from partial failure, while distributed inference can use it to assign model roles and coordinate intermediate artifacts. In both cases, NDN should provide the generic service semantics: request publication, provider metadata, provider selection, authorization, status diagnosis, and response verification. The application defines the domain-specific work; the framework defines how selected providers participate in one logical service workflow.

3.4 Generic Dynamic API

NDN v0.1 moved toward a generic dynamic API instead of generated, service-specific stubs. A provider can register a typed service handler:

```
provider.addHandler<RequestT, ResponseT>(
    ndn::Name("/ObjectDetection/YOLOv8"),
    handler);
```

A user can invoke a service through the generic runtime:

```
user.RequestService<RequestT, ResponseT>(
    providers,
```

```

ndn::Name("/ObjectDetection/YOLOv8"),
request,
onResponse,
onTimeout,
timeoutMs,
strategy);

```

This API is intended to keep application data in application-defined request and response objects while the framework handles naming, publication, permission checks, tokens, provider selection, and response delivery.

3.5 Service Message Model

NDNSF v0.1 uses four framework messages to separate service publication, provider admission, provider selection, and result delivery. This message model is central to the one-to-many service design: a request can be observed by multiple authorized providers, providers can expose availability and metadata before execution, and the user can explicitly bind the request to selected providers.

Table 3: NDNSF v0.1 service messages.

Message	Producer	Purpose
RequestMessage	User	Publishes a named service request and application payload.
RequestAckMessage	Provider	Reports whether the provider is willing to serve the request and exposes runtime metadata for selection.
SelectionMessage	User	Selects one or more providers and binds the request to the selected provider tokens.
ResponseMessage	Provider	Returns the service result from a selected provider.

Table 4: Representative fields carried by the four message types.

Message	Representative Fields
RequestMessage	service name, request ID, requester identity, payload, UserToken
RequestAckMessage	service name, request ID, provider identity, status, message, metadata payload, echoed UserToken, ProviderToken
SelectionMessage	service name, request ID, selected provider identity, ProviderToken
ResponseMessage	service name, request ID, provider identity, result payload, echoed UserToken

The message fields are intentionally service-independent. Application-specific data remains in the payload or in typed request/response objects, while the framework uses service names, request IDs, identities, and one-time tokens to provide authorization checks, replay resistance, and response verification.

3.6 Service Invocation Workflow

The normal NDNFSF invocation workflow is:

1. The user publishes a service request under a service name.
2. Providers with permission to serve the service receive the request.
3. Providers decide whether to ACK based on availability, local state, and application-defined metadata.
4. The user collects ACKs during an ACK window.
5. A selection strategy chooses one or more providers.
6. Selected providers execute the request and publish responses.
7. The user verifies tokens, provider identity, and response data.

This workflow supports both one-to-one and one-to-many service models. The same request can reach multiple providers, enabling the user to select the first available provider, randomly balance load, select all providers, or use a custom strategy based on metadata.

The implemented provider-selection strategies are deliberately generic. The *First Responding* strategy selects the first successful ACK, which is useful when low latency is the main goal. *Random Selection* chooses among valid ACK candidates, which can spread load without requiring detailed provider state. *All Selected* chooses every provider that returned a valid ACK, which supports cooperative or redundant execution. A *Custom Selection* strategy runs after the ACK collection window and can use ACK metadata such as latency, queue length, load, location, model availability, safety state, memory, or accelerator capacity. The key design point is that readiness reporting and final provider choice are separate framework steps: providers decide whether and how to ACK, and the user-side strategy decides which ACK candidates should be selected.

3.7 Security Model

NDNSF uses a controller-authorized permission model. The ServiceController issues encrypted permission responses to target identities. Users and providers fetch their respective permissions, and permission Data is signed by the controller. NAC-ABE attributes map service messages to service-level authorization namespaces. ABE-protected challenges, validated signatures, and invocation-state checks jointly enforce service-level authorization and reject replayed or mismatched Request/ACK/Selection/Response messages.

NAC-ABE enforcement is tied to the invocation tokens rather than to a separate permission list carried in every service message. A Request message includes a fresh `UserToken` encrypted under the service attribute, so only an authorized provider can decrypt the request and return that token. The ACK echoes the `UserToken` and carries a fresh `ProviderToken` encrypted under the permission attribute. The user proves selection authority by returning the selected provider's `ProviderToken` in the Selection message, and the final

Response echoes the `UserToken`. In this way, NAC-ABE attributes gate who can decrypt each authorization token, while one-time tokens bind that authorization check to a single service invocation.

Figure 4 illustrates the intended policy model: provider permissions specify which named services an entity may offer, while user permissions specify which named services an entity may access. This proposal keeps that service-name-centered policy model in the UAV and distributed-inference extensions, while adding deployment procedures that make the trust root and publisher namespaces explicit.

<pre> provider-policies { provider-policy { for /UAVNET/drone1 allow { /FlightControl/Takeoff /FlightControl/Land /ImageProc/YOLO12 } } provider-policy { for /UAVNET/drone2 allow { /FlightControl/Takeoff /FlightControl/Land /ImageProc/YOLO12 } } provider-policy { for /UAVNET/drone3 allow { /FlightControl/Takeoff /FlightControl/Land /Sensor/HyperSpectral } } } </pre>	<pre> user-policies { user-policy { for /UAVNET/gs/ allow { /UAVNET/drone1/FlightControl/Takeoff /UAVNET/drone1/FlightControl/Land /UAVNET/drone2/FlightControl/Takeoff /UAVNET/drone2/FlightControl/Land /UAVNET/drone3/FlightControl/Takeoff /UAVNET/drone3/FlightControl/Land } } user-policy { for /UAVNET/drone2/ allow { /UAVNET/drone3/Sensor/HyperSpectral } } user-policy { for /UAVNET/drone3/ allow { /UAVNET/drone1/ImageProc/YOLO12 /UAVNET/drone2/ImageProc/YOLO12 } } } </pre>
--	---

Figure 4: Example provider and user access-control policies in NDNSE.

The current model separates:

- **user authorization:** whether the user may request a service;
- **provider authorization:** whether a provider may serve a service;
- **message integrity:** whether a service message is signed and valid;
- **data confidentiality:** whether the payload is encrypted for authorized participants;
- **replay protection:** whether one-time tokens have already been consumed.

3.7.1 Rationale for ABE-Backed Service Authorization

NDNSF adopts ABE-backed authorization to support confidential discovery among authorized service Providers and to manage service permissions separately from identity-signing credentials. This is a choice for NDNSF's invocation workflow, not a claim that ABE is universally preferable to role-based authorization. Execution authorization determines whether an entity may invoke or provide a service; content confidentiality determines who can read protected message content. Signature validation, service permission checks, and invocation-state checks remain distinct requirements.

Confidential discovery. Before Provider selection, the User publishes a discovery descriptor containing the location, deadline, and resource requirements needed for a Provider's positive-ACK decision. For example, nearby UAVs need an observation area and arrival deadline to decide whether to participate, but do not yet need the full application input. ABE protects this descriptor for Providers authorized for the service without enumerating their identities. Confidentiality is against parties lacking the required decryption capability: authorized but unselected Providers can still read the descriptor. Content encryption does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, the full application input and its decryption material are restricted to selected Providers through Selection. A Trust Schema alone does not encrypt content; a role-authorized design can also use ABE or service-group encryption to provide discovery confidentiality.

Permission aggregation and credential reuse. In KP-ABE, ciphertext is associated with attributes and a decryption key (DKEY) encodes an access policy [7]. NDNSF associates Provider access with /SERVICE/<service> and User access with /PERMISSION/<service>. Within one authority and ABE parameter generation, it combines an entity's service entitlements into an OR policy for its individually issued DKEY, while reusing its identity-signing credentials. DNMP instead constrains command targets and actions through role signing keys and a Trust Schema [18]. Existing roles may cover several permissions; different entitlements may require changes to role assignments, credentials, or rules, not necessarily one new role per service.

For comparison, our per-service role-certificate alternative would issue each entity a certificate for each authorized service role. A User would use its service-role key to sign Request and Selection Data; a Provider would use its service-role key to sign ACK and Response Data. Validators would check the service, message type, certified role, certificate status, and invocation state; encryption would be supplied separately. ABE avoids these additional service-specific signing credentials, but still requires DKEY issuance and updates, with size and processing costs that can grow with policy complexity. Certificate-based designs can also reuse credentials; fewer credential objects alone do not establish lower total cost.

Why retain ABE-backed challenges? The Requester Challenge (UserToken in the implementation) and Provider Challenge (ProviderToken) check access to the required service and permission decryption capabilities in the current exchange. Signatures identify the participants, while matching and one-time-state checks reject mismatched or replayed messages. This reuses attribute-based permission management instead of adding service-role signing credentials. It is not cost-free: confidential discovery already needs Provider DKEYs, whereas User permission DKEYs and challenge processing are additional authorization costs. ABE confidentiality combined with role-certificate authorization remains a valid alternative.

3.7.2 *Runtime Permission Withdrawal*

The current runtime implements permission withdrawal through Controller-signed authorization status and checks at invocation processing points. A participant must retrieve and validate updated status before enforcing the newly withdrawn authority. ABE parameter rotation and refreshed DKEYs protect subsequent exchanges encrypted under the new generation. The present rotation is Controller-wide, so other authorized members may also need refreshed DKEYs; it is not a zero-fan-out revocation scheme.

DNMP describes encryption-key rotation to restrict subsequent access to measurement data, but does not detail a comparable online command-authorization revocation protocol [18]. A role-certificate comparison design can revoke the relevant credential and require execution nodes to check signed revocation status or reject expired certificates. This invalidates authority at the validator; it does not physically retrieve distributed certificates. Both approaches require update propagation, and neither retracts previously disclosed keys or plaintext.

Focused integration tests exercise NDNFSF's runtime revocation checks, including identity-, certificate-, and service-scoped withdrawal. These tests establish behavior at the exercised runtime boundaries, not network-wide instantaneous revocation or lower cost than certificate-based alternatives. The remaining evaluation should measure status-propagation delay, rejection delay, rekeying traffic, and effects on unaffected members. Runtime status validation and key rotation therefore extend NDNFSF to permission withdrawal, with explicit propagation and rekeying costs.

3.8 **Completed Foundation and Current NDNFSF Validation**

The NDNFSF evaluation compares NDNFSF against NSC and gRPC in MiniNDN. It covers service latency and success under wired loss, intermittent Providers, wireless mobility, adaptive admission, selective ACK, and Provider-selection scenarios. The current evidence supports three bounded multi-Provider claims: NDNFSF can discover Providers through a service namespace without a pre-registered address list, can reduce tail latency when mobility actually requires a

Provider switch, and can avoid redundant Provider execution. It does not claim that NDNFSF is universally more reliable or faster under mobility.

The strongest evidence comes from scenarios where endpoint binding or a single provider becomes fragile. In the lossy wired Memphis–CSU–UCLA path, packet round-trip loss is $1 - (1 - \ell)^4$ for per-link loss ℓ . NDNFSF completed 599 of 600 requests at 1% link loss, where the packet round-trip loss is 3.94%, and 598 of 600 requests at 3% link loss, where the packet round-trip loss is 11.47%. At 10% link loss, where packet round-trip loss rises to 34.39%, the best measured NDNFSF result was 539 of 600 requests. This identifies the bounded-time delivery limit of SVS-based service invocation under heavy loss, rather than a failure of the NDNFSF service model. In a wired multi-provider experiment with intermittent provider rejection, NDNFSF maintained 99.46% success at 20% provider-failure probability and 96.70% success at 30%, whereas the single-provider baselines dropped to roughly two-thirds success or lower. The wireless rows below remain historical diagnostics because their single-Provider baselines and timing provenance were not matched. The current fair campaign uses byte-identical traces, four Providers per system, common publication times, and matched deadlines. Its unconditional 50/100/150 m controls show that coverage dominates all-request success. In the registered ten-seed 100 m, 2 m/s holdout, the 1,312 requests unanimously classified as requiring a Provider switch show an NDNFSF seed-p95 latency reduction of 597 ms versus gRPC-SEQ-4 (95% CI $[-992, -200]$ ms) and 2,618 ms versus NSC-4 (95% CI $[-2916, -2315]$ ms). The success comparison does not establish superiority. In a separate transition test, NDNFSF completes 357/357 requests with zero pre-registered Provider endpoints after a new Provider becomes solely reachable; static gRPC/NSC complete 0/357, whereas their pre-registered variants complete 357/357. This establishes native service-namespaces discovery while recognizing that an external resolver can supply equivalent endpoint updates to gRPC or NSC. The adaptive-admission and Selective ACK experiments further show why the framework needs runtime metadata: admission control kept P95 latency below 270 ms under overload, and Selective ACK raised success from 1.82% to 100% when the nearest provider was overloaded. Together with the

capacity-matched six-seed repeat (NDNSF: 1,798/1,800 successes and 1,800 provider executions; gRPC-PAR-4: 1,800/1,800 successes and 7,180 server executions; seed-level bootstrap 95% interval $[-0.22, 0.00]$ pp), the evidence also supports a Provider-work-efficiency mechanism claim. These results jointly support native discovery, conditional switching latency, and execution efficiency, with explicit limits on general mobility claims.

Table 5: Selected reliability results from the submitted NDNSF v0.1 evaluation; wireless rows are historical diagnostics.

Scenario	Condition	NSC	gRPC	NDNSF
Intermittent wired providers	10% failure probability	83.25%	83.35%	99.82%
Intermittent wired providers	20% failure probability	66.58%	66.69%	99.46%
Intermittent wired providers	30% failure probability	49.92%	65.36%	96.70%
Wireless mobile providers	100 m AP range	45.42%	45.23%	96.25%
Wireless mobile providers	150 m AP range	68.75%	68.05%	100.00%
Wireless mobile providers	200 m AP range	88.33%	87.14%	100.00%

Table 6: Selected mechanism results from the submitted NDNSF v0.1 evaluation.

Mechanism	Baseline Behavior	NDNSF Behavior
Adaptive admission under 100 RPS offered load	Without admission control, P95 latency reached 4254.39 ms and timeout rate reached 3.55%.	With admission control, admitted rate was limited to about 81 RPS, P95 latency stayed at 268.12 ms, and timeout rate was 0.00%.
Selective ACK with nearest provider overloaded	ACK-all selected the nearest overloaded provider for 2209 of 2250 requests and achieved 1.82% success.	Selective ACK suppressed overloaded ACKs, shifted selection to available providers, and achieved 100.00% success.
Real-machine intermittent availability	Single-provider NSC/gRPC stayed near 50% success when the chosen provider was unavailable for half the run.	NDNSF selected among two providers and reached 74.62% success at 10 RPS and 73.73% at 30 RPS, close to the expected availability window.

3.9 Limitations of NDNSF v0.1

NDNSF v0.1 establishes feasibility, but it does not yet fully address the requirements of dynamic service applications beyond controlled framework tests. The main limitations that motivate the remaining dissertation work are:

- **Low-latency known-provider control:** the normal Request/ACK/Selection/Response workflow is appropriate for provider discovery, but control commands to a known drone or device need a faster Targeted path while preserving authorization and replay protection.

- **Provider collaboration:** v0.1 supports selecting providers, but larger tasks need collaboration among multiple providers, partial failure handling, and compensation requests.
- **Service-container composition:** real applications often contain multiple services in one trusted process. Those services need efficient local composition without exposing a network bypass to remote callers.
- **Large artifacts:** v0.1 service payloads are not intended to carry models, recordings, images, or intermediate tensors. Applications need typed references to named Data objects plus native NDN segmented retrieval for large signed and encrypted objects.
- **Operational validation:** v0.1 demonstrates the security model, but broader validation needs clearer certificate bootstrapping, trust-schema guidance, configuration procedures, and reproducible experiments.
- **Application-level validation:** v0.1 evaluation shows the core framework works in MiniNDN, but the dissertation must evaluate whether the design supports demanding UAV and distributed-inference workflows.

3.10 Supporting Mechanisms for Provider Collaboration

The following mechanisms support provider collaboration in specific operating conditions. They are motivated by the UAV and distributed-inference workloads, but they are not separate dissertation threads and should remain generic framework mechanisms rather than application-specific shortcuts.

3.10.1 Targeted Invocation

Some services, such as UAV flight-control commands, are directed at a known provider and must minimize latency. The proposed Targeted invocation path keeps NDNSF permissions,

one-time user/provider tokens, and replay protection, but avoids repeated ACK/Selection rounds after a token batch has been bootstrapped through the normal flow. This gives known-provider commands a fast path while keeping the security semantics of normal NDNSF invocation.

3.10.2 *Trusted Local Invocation*

A service-container application may contain several services in the same trusted process. A local call from one internal service component to another should not need to publish an NDN request. The proposed local invocation path therefore provides synchronous and asynchronous in-process invocation for explicitly registered local services. It is not a wire-protocol mode: remote callers cannot request it, and it bypasses NDNSF network security only inside the trusted local container boundary.

3.10.3 *Large-Data Reference Abstraction*

Models, video, images, activations, and logs should not be placed in small request payloads. The proposed pattern is to let the producer segment the object as named NDN Data and place only a typed object reference in the NDNSF request or response. The reference can include the object name, object type, version or session id, expected digest, expected size, final segment information, source type, and optional encryption metadata. The receiver then uses normal NDN segmented fetching, such as `ndn-cxx` segment fetch logic, to retrieve the object by name.

This abstraction is intended to be a generic NDNSF mechanism, not an API for a particular storage service. Small request and response payloads remain useful for control messages and compact typed objects. When the payload becomes a large artifact, the same logical service invocation should carry a *reference* rather than the artifact bytes. The application or helper can turn a payload into named Data segments, sign and optionally encrypt those segments, and then pass a typed reference through NDNSF. This keeps service selection, authorization, token checks, and timeout diagnosis in the normal NDNSF path, while large bytes move through NDN's native named-data retrieval path.

The same reference format should be usable across UAV recordings, camera artifacts, model shards, runtime artifacts, intermediate activations, and logs. As a result, UAV-APP and DistributedInference can share one framework-level large-data vocabulary: object type, object name, producer, session or version, size, digest, final segment hint, and optional encryption metadata. This avoids embedding application-specific conventions such as raw Data-name strings inside service payloads and makes large-object behavior inspectable by applications that do not understand NDN packet details.

This design can optionally use persistent storage, but storage is an implementation support component rather than a separate research direction in this proposal. If persistent storage is used, it stores opaque NDN Data segments that have already been named, signed, and optionally encrypted by the application or by a framework helper. Security is not handled by a storage-specific path: NDNSF's normal hybrid encryption, signatures, tokens, permissions, and trust-schema validation protect the service message and the referenced Data. Discovery metadata can still accelerate validation, but large-object transfer itself is based on typed object references and native NDN segmented retrieval.

This chapter supports RQ2 by previewing the reusable NDNSF mechanisms that the dissertation will refine: secure invocation, provider authorization, provider collaboration, and its supporting mechanisms. It supports RQ3 by identifying where collaboration introduces reliability, security, and latency tradeoffs, and where supporting mechanisms are needed to control overhead without weakening the core service semantics.

Chapter 4

UAV Network Application over NDNSF

4.1 Why UAV Networks Need a Service Framework

UAV applications are a strong test case for NDNSF because they combine mobility, real-time control, video streaming, distributed sensing, multi-agent coordination, and safety constraints. A UAV network may include a ground station, several drones, cameras, flight controllers, object-detection services, storage capabilities, and mission-planning components. Each component exposes capabilities that naturally look like services.

For the dissertation, the UAV workload is specifically used to stress provider collaboration under mobile and command-sensitive conditions. A ground station may need to discover multiple drones, select one or more providers for a mission, issue targeted commands to known drones, compensate for a failed mission segment, and retrieve video or recording objects without treating large data as RPC payload. This makes UAV-APP a validation workload for the NDNSF framework: it asks whether provider collaboration, authorization, selection status, and supporting low-latency or large-data mechanisms can work together in an operator-facing service container.

Developing such an application over IP creates several recurring problems:

- **Address binding.** A drone's identity and its IP address may change as it moves, reconnects, or switches networks.
- **NAT and mobility.** Direct connectivity to drones is not always available, especially in ad hoc or field deployments.
- **Service discovery.** A ground station needs to know which drones offer camera, telemetry, flight-control, and mission services.
- **Command authorization.** Flight-control commands must be restricted to authorized users and protected against replay.

- **Data verification.** Telemetry, images, videos, and mission results should be verifiable independently of the channel.
- **Failure recovery.** If one drone disconnects or fails a task, the mission should compensate using other drones when possible.
- **Large data.** Live video and recorded video should use streaming and segmented data retrieval instead of small RPC payloads.

4.2 UAV Services

The proposed NDN-SF-UAV-APP models a UAV network as a service-container application. The ground station and each drone run containers with different service instances, and the APP-level configuration determines identities, service names, runtime roles, camera behavior, and flight-controller backend. Table 7 summarizes the service types.

Table 7: Service types in the proposed UAV application.

Service Type	Purpose
Flight control	Arm, takeoff, land, goto, manual control, and generic MAVLink execution.
Telemetry and state	Report GPS, altitude, battery, armed state, flight mode, readiness, camera availability, link state, and mission state.
Video control	Start/stop live video, negotiate bitrate, and report actual streaming state.
Video data	Publish encoded frames or chunks as NDN Data for low-latency playback.
Recording discovery	Authorized discovery of locally recorded encrypted video objects.
Recording retrieval	Retrieve encrypted signed NDN Data objects through large-data references.
Object detection	Detect objects such as cars and trucks from camera frames, either at the ground station or onboard.
Mission/patrol	Assign waypoints, partition tasks among drones, start/stop missions, and compensate for missing subtasks.
Discovery/status	Publish installed service information and readiness state.

4.3 Architecture

The UAV application contains two main roles:

- **Ground station role:** operator interaction, map-based mission planning, telemetry and video observation, object-detection service composition, and NDN SF user behavior.
- **Drone role:** flight-control adaptation, telemetry publication, camera and video publication, encrypted recording support, mission execution, and NDN SF provider behavior.

The application is explicitly service-oriented. A drone does not expose one monolithic endpoint. Instead, it exposes independently named services for flight control, telemetry, video control, camera frames, recording discovery, and mission assignment. The ground station uses structured runtime state to represent telemetry freshness, mission progress, video availability, and safety readiness, so that operator-visible status and command decisions remain consistent across independently named services. Known-drone commands use Targeted invocation, while collaborative workflows such as patrol assignment use normal NDNFSF selection.

4.4 Operator Workflow and State Discipline

The UAV application is not intended as a replacement for an existing ground control station. Its purpose is to validate whether NDNFSF can support a real UAV service workload. For that validation to be meaningful, the GS must still answer the operator questions that matter in practice: which vehicle is selected, whether it is ready and safe to control, which command or mission is active, whether telemetry and video are fresh, and what will happen if a link or device fails. The proposed UAV work therefore treats the GS as a state-driven operator console, not only as a collection of service invocations.

Table 8: Operator-facing state discipline planned for NDNSF-UAV-APP.

State Area	Planned Behavior
Vehicle selection	The GS keeps the operator’s selected vehicle stable while telemetry, video, and service updates arrive asynchronously.
Readiness	The GS presents an integrated readiness view for camera, flight controller, telemetry, mission, video, recording, and storage-related capabilities.
Telemetry freshness	The GS distinguishes current, stale, and missing telemetry so that old data is not mistaken for current state.
Command lifecycle	Control and mission commands expose observable progress, completion, and timeout outcomes instead of opaque success/failure messages.
Mission progress	Patrol missions expose assignment, progress, partial completion, cancellation, and compensation behavior across drones.
Failure behavior	Timeouts and device/link failures surface diagnostic information so that the operator or application policy can decide the next action.

4.5 Live Video and Recording

Live video is initiated by a GS command to a drone-specific video-control service. The drone negotiates the accepted bitrate and frame size, encodes camera frames as a streaming codec, and publishes stream Data under names that include a stream identifier and monotonically increasing packet sequence. Each packet also carries stream metadata so the receiver can reject stale packets from a previous stream. The GS fetches and decodes video with an adaptive low-latency policy that uses observed RTT, bitrate, timeout pressure, decoder backlog, and published/decoded gap to tune the prefetch window and stale-frame skipping policy.

Recording is intentionally different from live control. The drone may keep its camera

active and record locally even when live streaming is off. The recorded object is represented as a large-data reference to signed and encrypted Data segments. The ground station discovers available recordings through an authorized service invocation, then fetches the referenced segments by name and decrypts locally. Thus storage does not need to understand service authorization; authorization and key material are handled by NDN SF service messages, while replay or stale-data problems are handled by session identifiers, stream identifiers, and signed Data names.

4.6 Flight Control and Safety

Flight-control commands are naturally targeted at a specific drone. The proposed work uses Targeted NDN SF invocation for commands such as arm, takeoff, land, goto, emergency stop, and manual control. The drone-side adapter supports mock mode for MiniNDN tests and MAVLink/PX4 mode for SITL or real flight controllers [22, 23]. Manual-control commands are continuous control messages rather than mission services; their responses can carry telemetry so the GS can update altitude, battery, GPS, speed, controller state, and flight-safety indicators while the operator is flying.

Safety mechanisms include:

- readiness gates based on heartbeat, GPS/EKF, battery, mode, and arming checks;
- command acknowledgments from the flight controller;
- manual-control timeouts that stop sending stale control values;
- emergency stop or hold/land behavior after communication failure;
- explicit warnings that real propeller tests require a validated safety procedure.

4.7 Multi-UAV Patrol

The proposed patrol service demonstrates provider collaboration. A ground station creates a waypoint mission and partitions it among two or more drones. The design clusters waypoints according to the number and position of available drones so each drone receives a compact region and returns to its own departure point after completing its segment. Each drone receives a mission segment, executes it through MAVLink, and returns status. If a drone fails or returns an incomplete result, the GS can issue a compensation request for the missing segment rather than restarting the entire mission.

This design illustrates a key NDN SF principle: compensation requests are application-level parts of the same logical mission, while NDN SF provides the secure service invocation, provider selection, and message verification substrate.

The current UAV-APP is a research prototype, not yet a field-ready or safety-certified ground-control system. This distinction is important for the proposal: practical use, SITL, and selected real-machine testing may expose new requirements or changes in command handling, telemetry freshness, video behavior, safety policy, deployment, or operator workflow. Those discoveries are part of the research value because they can feed back into NDN SF's framework mechanisms.

4.8 Evaluation Plan

The UAV application will be evaluated in three stages:

1. **MiniNDN regression tests:** controller, one or more drones, operator console behavior, video start/stop, encrypted recording playback, and patrol service execution.
2. **SITL tests:** PX4/jMAVSIM control for arm, takeoff, land, manual control, and mission execution.
3. **Real-machine validation:** one PC and one or more drones with NFD routes, certificates, camera devices, and MAVLink connections.

Metrics will include command latency, video startup time, decoded frame rate, recording retrieval time, mission completion rate, compensation overhead, telemetry freshness, and failure recovery time.

This chapter supports RQ4 by defining UAV as the command-sensitive, real-time, mobile workload used to validate NDNSF beyond core framework tests. It also feeds back into RQ2 and RQ3 by using UAV behavior to stress Targeted invocation, provider collaboration, timeout diagnosis, and large-data references.

Chapter 5

Distributed AI Inference over NDNF

5.1 Motivation

Distributed AI inference is another strong test case for NDNF. AI models are large, runtime artifacts are sensitive, intermediate tensors can be large, and inference providers may be heterogeneous. A model may be split across edge devices, drones, ground stations, GPUs, and cloud resources. The user wants to invoke an inference service by model or task name, while the system determines where model shards and stages should run.

For the dissertation, distributed inference stresses provider collaboration in a different way from UAV workloads. Providers are not collaborating because they are vehicles in a shared physical mission; they collaborate because a single inference task may require multiple model chunks, runtime artifacts, activation exchanges, and dependency-ordered execution steps. This workload therefore tests whether NDNF can keep collaboration generic: the framework should coordinate authorized providers, large-data references, status diagnosis, and result verification, while the inference layer supplies the model-specific dependency graph and execution plan.

In IP-based systems, distributed inference often relies on custom orchestration: host addresses for model stages, separate object stores for model artifacts, custom integrity checks, ad hoc retry behavior, and manual placement policies. This makes it hard to build applications that remain correct when providers join, leave, fail, or move.

5.2 Approaches to Distributed AI Inference

The dissertation will review the main distributed inference approaches:

- **Replication/data parallel inference:** multiple providers hold the same model and serve independent requests.
- **Pipeline partitioning:** model stages execute sequentially on different providers,

following split-computing ideas such as DNN partitioning across mobile, edge, and cloud resources [24, 25].

- **Layer or operator partitioning:** the model graph is split at finer granularity according to compute and memory constraints.
- **Early-exit inference:** computation can stop early when a confidence threshold is reached [26].
- **Collaborative inference:** multiple providers cross-check, aggregate, or vote on results.
- **Edge-cloud split inference:** lightweight preprocessing happens at the edge while heavier stages run on ground-station or cloud nodes.

This proposal uses unambiguous partition terminology. A *pipeline/layer-range partition* assigns contiguous model-layer ranges to sequential stages. An *intra-layer/tensor partition* shards the work of one stage across multiple tensor ranks. A heterogeneous hybrid layout is written as $N \times \{M_i\}$, where N is the number of pipeline stages and M_i is the number of tensor ranks used by stage i . The values of M_i need not be equal. Tensor partitioning is allowed only when the model adapter certifies the required collective and redistribution semantics; aggregate memory across unrelated devices must not be treated as capacity for an unsplittable role.

5.3 NDNSF-Based Inference Architecture

The public application call identifies the model, task, input, options, and deadline. It does not provide a pre-split deployment layout or a list of Provider roles. NDNSF Core remains a model-neutral carrier for Request/ACK/Selection/Response and named large-data references. NDNSF-DI owns model-graph interpretation, pipeline and tensor semantics, placement, assembly specifications, device bindings, and execution dependencies. The Repo stores immutable manifests and canonical layer bytes, not request-specific role shards.

The design distinguishes three identities that must never be collapsed:

- **canonical layer identity**, which names reusable model-layer bytes and their immutable manifest;
- **assembled fragment identity**, which records the ordered layers, adapter, assembly options, and logical role produced locally by one Provider;
- **loaded runtime identity**, which additionally binds the assembled fragment to a runtime, visible device set, and load configuration.

Planning occurs only after the immutable ACK collection closes. Signed ACK offers expose Provider load, network observations, canonical-layer cache state, and explicit per-device memory/topology information. Zero visible accelerators is a valid CPU-only offer. ACK collection and queue acceptance do not reserve accelerators; device feasibility is revalidated immediately before loading. For ONNX models [27], the proposed path is shown in Figure 5.

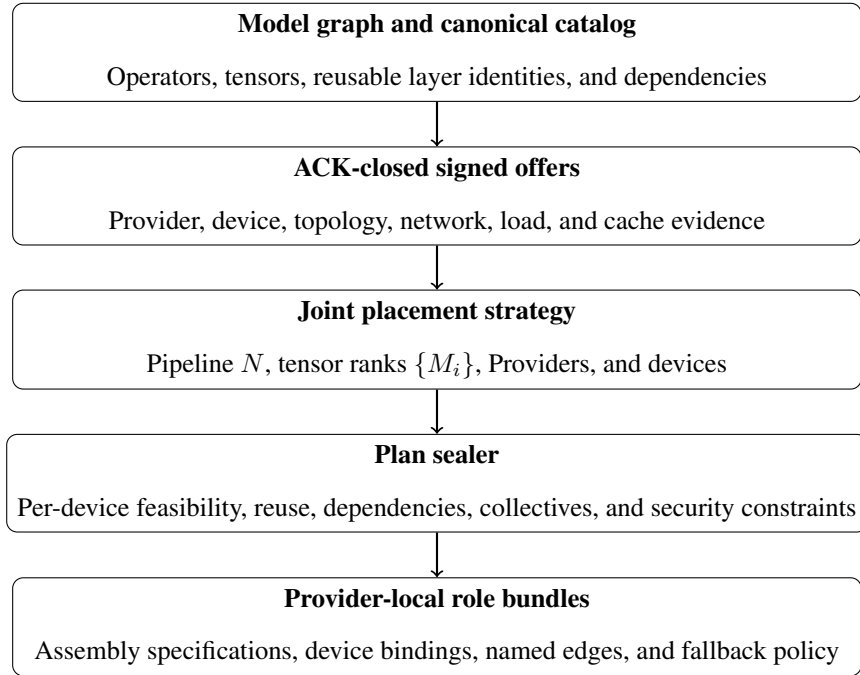


Figure 5: Late-bound generation path from a model graph, canonical-layer catalog, and ACK-closed Provider offers to sealed Provider-local execution bundles.

The same interface can support non-ONNX models through a certified model adapter. After final Selection, each selected Provider fetches missing canonical layers, assembles its

assigned fragment locally, revalidates the selected CPU/SINGLE.DEVICE/DEVICE.SET binding, loads the runtime, and becomes locally ready. Execution is then driven by named dependency and collective edges; it does not require a global all-Providers-ready barrier.

5.4 Automatic Collaboration Plan Generation

A major proposed contribution is automatic generation of collaboration plans from AI model structure and resource information. The input includes:

- the model graph, canonical-layer catalog, tensor dependencies, adapter-certified partition capabilities, and estimated activation or collective sizes;
- an immutable ACK-closed set of signed Provider offers containing explicit CPU and per-device memory/topology state;
- network latency, RTT, throughput, and relevant topology observations;
- canonical-layer, assembled-fragment, and loaded-runtime cache state;
- authorization, naming, dependency-edge, and key-scope constraints.

The output is a sealed collaboration plan containing logical roles and ranks, Provider-specific assignments, `RoleAssemblySpecs`, `explicit DeviceBindings`, named dependency/collective edges, and the evidence used by the decision. A reuse-aware strategy may prefer Providers that already hold required canonical layers, but reuse never overrides model compatibility, per-device feasibility, topology, authorization, or deadline constraints.

5.5 Dependency-Driven Execution

The planned runtime should hide dependency management from AI application developers. A Provider should not contain YOLO-specific or fully-connected network-specific communication code. It receives a Provider-local role bundle, assembles and loads the specified

fragment, consumes declared input edges, executes its pipeline stage or tensor rank, performs certified collectives or redistributions, and publishes only the outputs authorized by its edges. This executor model is intended to support chains, fan-in/fan-out graphs, and heterogeneous $N \times \{M_i\}$ layouts. Direct and cached Data must pass the same producer, name, dependency-edge, and key-scope checks.

5.6 Current Prototype Status and Evidence Boundary

The available measured evidence predates the V3 reusable-layer architecture. The NativeTracer campaign uses a small Qwen-style ONNX artifact and an earlier Backbone/Head/Merge role/chunk workflow over the full MiniNDN network harness. It exercises real model execution, Repo-backed artifact fetching, service manifests, NDNSF large-data references, and named intermediate tensor objects. It does not validate canonical-layer reuse, Provider-local assembly identity, tensor/hybrid execution, multi-accelerator placement, or just-in-time device admission.

The current layout campaign compares two executable runtime layouts under the same full-network harness. In ten successful runs per layout, the shared-backbone layout had mean latency 259.4 ms, p50 267.7 ms, p95 283.9 ms, and standard deviation 22.1 ms. The single-provider layout had mean latency 186.8 ms, p50 191.7 ms, p95 206.3 ms, and standard deviation 20.0 ms. For this small model, single-provider execution is faster because it avoids the fixed cost of cross-provider dependency exchange. This is a useful negative result: NDNSF-DI should not split a model merely because multiple providers are available; the planner must compare compute cost, intermediate-data size, provider capacity, RTT, bandwidth, and artifact placement before selecting a layout.

Earlier YOLO-style and small fully connected examples remain useful regression cases for dependency-driven execution. The V3 architecture described above is therefore presented as a design and implementation target whose evidence gates remain open, not as a capability established by this campaign. The next work is to remove request-specific shard

publication, qualify canonical-layer reuse and Provider-local assembly, and determine when pipeline/tensor/hybrid execution outweighs its coordination cost.

5.7 Prototype and Evaluation Plan

The evaluation will proceed from deterministic integration tests to MiniNDN and sealed real-model/container qualification. Planned cases include:

- exact canonical-layer, assembled-fragment, and loaded-runtime identity derivation, including cold, warm, restart, and cache-corruption cases;
- CPU-only, zero-visible-accelerator, single-device, and multi-device Provider offers with per-device feasibility and topology revalidation;
- proof that ACK collection and queue acceptance do not reserve devices, followed by just-in-time admission success and failure cases;
- pipeline-only, adapter-certified tensor, and heterogeneous $N \times \{M_i\}$ hybrid layouts, including collective and redistribution dependencies;
- direct versus cached Data with positive and negative producer, name, dependency-edge, assignment, and key-scope checks;
- local baseline versus NDN-SF execution, cold versus reused artifacts, Provider failure/replanning, and output consistency;
- integration-harness qualification followed by MiniNDN and sealed container/Tiger runs for representative real models.

Metrics will include end-to-end latency, canonical-layer retrieval and local assembly time, load/admission time, cache reuse, peak per-device memory, activation and collective bytes, Provider utilization, success and recovery rates, output consistency, p50/p95 latency, variance, and planner prediction error.

This chapter supports RQ5 by defining distributed inference as the artifact-heavy, dependency-driven workload used to validate NDNF under multi-stage compute/data collaboration. It also feeds back into RQ2 by testing whether large-data references, ACK-closed planning, Provider-local bundles, named dependency edges, and long-running status can remain generic rather than model-specific.

Chapter 6

Research Plan and Timeline

6.1 Completed Work

The completed work includes the NDN service framework survey, NDNSF v0.1, core security design, provider selection, adaptive admission control, and preliminary MiniNDN evaluation. NDNSF v0.1 is not outside the dissertation; it is the completed first-stage core contribution on which the rest of the dissertation is built. Work after v0.1, including UAV-APP, DistributedInference, provider collaboration, and the supporting mechanisms needed to exercise it in realistic workloads, is treated as the application-driven extension and validation agenda to be completed and evaluated. Repository support may be used as an implementation component for large objects, but it is not presented as a separate dissertation thread.

6.2 Planned Work

The remaining dissertation work is now primarily a stabilization, integration, and evaluation effort organized into four threads:

1. **Framework extensions:** finalize provider collaboration as the central NDNSF extension, and stabilize supporting mechanisms for low-latency Targeted invocation, trusted local composition, large-data references, and NAC-ABE operational support, with emphasis on correctness, documentation, and regression coverage.
2. **UAV application:** stabilize the GS operator workflow, service-container organization, MAVLink/PX4 integration, live video, encrypted recording, object detection, and multi-UAV patrol as an application-driven validation of NDNSF.
3. **Distributed inference:** complete the NativeTracer/ONNX execution path, generated service plans, reference-based artifact exchange, provider assignment, layout campaigns, and failure compensation, then freeze the evaluation scenarios.

4. **Evaluation and writing:** run final MiniNDN, SITL, selected real-machine experiments while converting the resulting evidence into dissertation chapters.

6.3 Evaluation Matrix

The final evaluation should connect the framework thesis to both application domains and show how the applications validate and extend NDNSF mechanisms. Table 9 summarizes the planned evidence.

6.4 Timeline

6.5 Risks and Mitigation

- **Dependency risk:** UAV and AI applications depend on many libraries. Mitigation: controlled dependency environments and container-based tests.
- **Real flight risk:** physical UAV testing is safety-critical. Mitigation: start with MiniNDN and SITL, then use controlled real-machine tests without propellers before flight.
- **Performance risk:** security and sync overhead may affect latency. Mitigation: use Targeted invocation, local invocation, and large-data references as supporting mechanisms where the collaborative path would otherwise add avoidable overhead, together with sampled logging and careful benchmark profiles.
- **Scope risk:** UAV and AI inference are both large applications. Mitigation: implement minimal convincing prototypes first, then extend only the features needed to demonstrate the framework.

This chapter connects the research questions to the dissertation execution plan. It shows that each RQ has a corresponding evidence path, while leaving room for application-driven findings from UAV and DistributedInference to refine the final NDNSF design.

Table 9: Evaluation matrix for the dissertation.

RQ	Dissertation Component	Evaluation Target	Evaluation Methodology
RQ1	Literature review	NDN service framework gap	Related-work comparison of RPC, robotics middleware, pub/sub, workflow systems, NDN service approaches, UAV networking, and distributed inference systems, summarized as framework requirements.
RQ2, RQ3	NDNSF core	Security, selection, targeted invocation, local composition, collaboration, and large-data support	MiniNDN regression and performance tests for permissions, NAC-ABE token routing, replay protection, provider selection, collaboration status, Targeted invocation, local invocation, and large-data references.
RQ4	UAV workload	Mobility, freshness, command lifecycle, media objects, and multi-UAV missions	GS/Drone MiniNDN tests, SITL MAVLink checks, command lifecycle logs, video stability, telemetry freshness, recording retrieval, mission progress, and failure recovery.
RQ5	DI workload	Model/data retrieval, split inference, dependency execution, planner decisions, and multi-provider collaboration	MiniNDN cluster tests for local-versus-distributed output equality, ONNX graph summary, generated policy checks, reference-based artifact provisioning, activation exchange, layout campaigns, provider utilization, and provider-failure cases.
RQ2, RQ4, RQ5	Operations	Reproducible deployment and selected real-machine validation	Certificate and route procedures, configuration checks, reproducible experiment setup, and selected real-machine tests.

Table 10: Proposed dissertation timeline.

Period	Planned Work
August 31, 2026	Stabilize NDNsf provider collaboration and its supporting Targeted, local, and large-data mechanisms; freeze interfaces, reproducible builds, regression coverage, and the experiment plan.
September 30, 2026	Complete the UAV MiniNDN and SITL evaluation for command lifecycle, telemetry readiness, live video, recording, mission workflows, compensation, and recovery.
October 30, 2026	Complete the distributed-inference evaluation, including ONNX policies, artifact and activation exchange, dependency-driven execution, output consistency, and Provider-failure recovery.
November 13–December 4, 2026	Freeze the accepted datasets, configurations, tables, and figures by November 13 while dissertation writing proceeds in parallel; deliver the complete chair-review draft by December 4.
January 15–February 5, 2027	Circulate the committee-ready draft by January 15, resolve chair comments, complete graduation and candidacy filings by February 5, and announce the defense at least three weeks before it is held.
By mid-February 2027	Hold the dissertation defense after committee review and the required public-notice period.
February 26, 2027	Complete post-defense corrections and obtain final committee approval.
March 5–26, 2027	Target internal ProQuest submission by March 5 and meet the official March 26 deadline for Spring 2027 graduation.

Chapter 7

Conclusion

This proposal presents NDN SF as a data-centric service framework for dynamic, secure, and collaborative networked applications. NDN SF v0.1 demonstrates that NDN can support a generic service abstraction with authorization, provider selection, and adaptive behavior. The proposed work centers the next framework step on provider collaboration, supported by Targeted invocation, trusted local composition, and large-data references, and validates this direction through two demanding application domains: UAV networks and distributed AI inference.

The expected outcome is not merely another application implementation, but a framework and design methodology for building service-oriented systems where services, data, permissions, and collaboration plans are named and verifiable. UAV-APP and DistributedInference are important because they stress NDN SF in different ways and reveal what the framework must provide, but the dissertation contribution remains the NDN SF framework, its mechanisms, and the evidence that those mechanisms generalize beyond a single application.

References

- [1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, B. Zhang, Named data networking, in: ACM SIGCOMM Computer Communication Review, 2014, pp. 66–73.
- [2] C. Yi, A. Afanasyev, I. Moiseenko, L. Wang, B. Zhang, L. Zhang, A case for stateful forwarding plane, Computer Communications 36 (7) (2013) 779–791.
- [3] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.
- [4] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.
- [5] Statevectorsync: A library for distributed real-time applications over ndn, <https://github.com/named-data/ndn-svs>, , accessed 2026.

- [6] ndn-cxx: Ndn c++ library with experimental extensions, <https://github.com/named-data/ndn-cxx>, , accessed 2026.
- [7] S. Dulal, T. Yu, S. Liu, A. R. Thieme, L. Zhang, L. Wang, Enhancing NAC-ABE to support access control for mHealth applications and beyond, arXiv:2311.07299 (2023). doi:10.48550/arXiv.2311.07299.
URL <https://arxiv.org/abs/2311.07299>
- [8] grpc: A high performance, open source universal rpc framework, <https://grpc.io/>, , accessed 2026.
- [9] R. T. Fielding, Architectural styles and the design of network-based software architectures, Ph.D. thesis, University of California, Irvine (2000).
- [10] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Ros: an open-source robot operating system, ICRA Workshop on Open Source Software.
- [11] Ros 2 documentation, <https://docs.ros.org/>, , accessed 2026.
- [12] V. John, X. Liu, A survey of distributed message broker queues, arXiv preprint arXiv:1704.00411.
- [13] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.
- [14] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.
- [15] M. A. ur Rehman, B.-S. Kim, Cfec: An ultra-low latency microservices-based in-network computing framework for information-centric iovs, IEEE Transactions on Services Computing 17 (6) (2024) 3199–3212.
- [16] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.
- [17] T. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, in: NDN Technical Report NDN-0074, Revision 1, 2021, pp. 1–20.
- [18] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122.
- [19] M. Imran, et al., Mia-ndn: Microservice-centric interest aggregation in named data networking, Sensors 23 (3) (2023) 1411.
- [20] R. Tourani, et al., Towards security-as-a-service in multi-access edge, in: Proceedings of the 4th ACM/IEEE Symposium on Edge Computing, 2019, pp. 358–363.

- [21] V. Goyal, O. Pandey, A. Sahai, B. Waters, Attribute-based encryption for fine-grained access control of encrypted data, in: Proceedings of the 13th ACM Conference on Computer and Communications Security, 2006, pp. 89–98.
- [22] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, M. Khalgui, Micro air vehicle link (mavlink) in a nutshell: A survey, *IEEE Access* 7 (2019) 87658–87680. doi:10.1109/ACCESS.2019.2924410.
- [23] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.
- [24] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697.3037698.
- [25] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017.226.
- [26] S. Teerapittayanon, B. McDanel, H. T. Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: 2016 23rd International Conference on Pattern Recognition, 2016, pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.
- [27] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.